Reproducing U-Net for Biomedical Image Segmentation on a Single Free-Tier GPU

Shubhro Shekhar Das
Rajiv Gandhi Institute of Petroleum Technology

Deep Learning course project

November 12, 2026

***Abstract*— U-Net [1] segments biomedical images with a contracting path, a symmetric expanding path, skip connections, and a loss that weights the thin gaps between touching cells. I re-implement it in PyTorch from the paper and reproduce the paper's Table 2 (Cell Tracking Challenge 2D datasets PhC-U373 and DIC-HeLa) at reduced scale on a free Colab T4 GPU. Because the challenge test labels are hidden, I evaluate on held-out annotated frames from a second sequence, so results are not directly comparable with the paper. On PhC-U373 the model reaches a SEG score of 0.772 (paper: 0.920); on DIC-HeLa the foreground IoU is high (0.826) but SEG collapses to 0.164, which I attribute, without having verified it, to touching cells merging during instance extraction. An ablation on PhC-U373 shows a small gain from elastic deformation and, contrary to the paper's motivation, no gain from the weight map. I discuss what these single-seed results can and cannot support.**

***Index Terms*—** U-Net, semantic segmentation, data augmentation, weighted loss, Cell Tracking Challenge, reproducibility

I. Problem and Motivation

Image classification networks output one label per image, but biomedical tasks need a label for every pixel, i.e. *localization*. Annotated biomedical data is also scarce: thousands of labeled images are usually out of reach [1], whereas the breakthrough of deep networks on ImageNet relied on a million labeled images [4].

The standard answer at the time was the sliding-window network of Ciresan et al. [2], which classifies the centre pixel of a local patch. It has two drawbacks. First, it is slow, since the network is rerun for every patch and overlapping patches share most of their content. Second, it trades localization accuracy against context: large patches need more max-pooling, which blurs the location, while small patches see little context.

U-Net builds on the fully convolutional network [3]. It keeps good localization and context at the same time, runs one forward pass per image tile, and trains from very few annotated images by leaning on augmentation. A further difficulty in cell segmentation is that objects of the same class touch; the paper addresses this with a pixel-wise weighted loss. This report reimplements these ideas and checks how far a short, single-GPU run gets toward the paper's reported numbers.

II. Method

A. Architecture

The network (Fig. 1) has a contracting path and an expanding path. The contracting path repeats two unpadded 3×3 convolutions with ReLU, followed by 2×2 max-pooling with stride 2; the channel count doubles at each step ($64, 128, 256, 512$, and 1024 at the bottom). Each expanding step applies a 2×2 up-convolution that halves the channels, concatenates the corresponding feature map from the contracting path, and applies two 3×3 convolution+ReLU layers. A final 1×1 convolution maps the 64-dimensional feature vector to the class scores. There are no fully connected layers.

Because the convolutions are *unpadded*, each loses two pixels in both dimensions. For the default depth, a 572×572 input yields a 388×388 output, i.e. a constant border of 92 px is lost on each side. The skip-connection features from the contracting path are therefore larger than the upsampled ones and are centre-cropped before concatenation. Every max-pool must see an even size, which constrains the input size; 572 satisfies it. In total the network has 23 convolutional layers (19 convolutions and 4 up-convolutions) and 31.0 M parameters.

B. Loss and weight map

The network outputs scores $a_k(\mathbf{x})$ for K classes at each pixel $\mathbf{x} \in \Omega$. A pixel-wise soft-max gives class probabilities

$$p_k(\mathbf{x}) = \frac{\exp\big(a_k(\mathbf{x})\big)}{\sum_{k'=1}^{K} \exp\big(a_{k'}(\mathbf{x})\big)}, \tag{1}$$

and training minimises the weighted cross-entropy

$$E = -\sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log p_{\ell(\mathbf{x})}(\mathbf{x}), \tag{2}$$

where $\ell(\mathbf{x})$ is the true label. The weight map is precomputed from the ground-truth instances:

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \exp\left(-\frac{\big(d_1(\mathbf{x}) + d_2(\mathbf{x})\big)^2}{2\sigma^2}\right), \tag{3}$$

with $w_0 = 10$ and $\sigma \approx 5$ px. Here w_c compensates class frequencies, and d_1, d_2 are the distances to the nearest and second-nearest cell. Between two touching cells both distances are

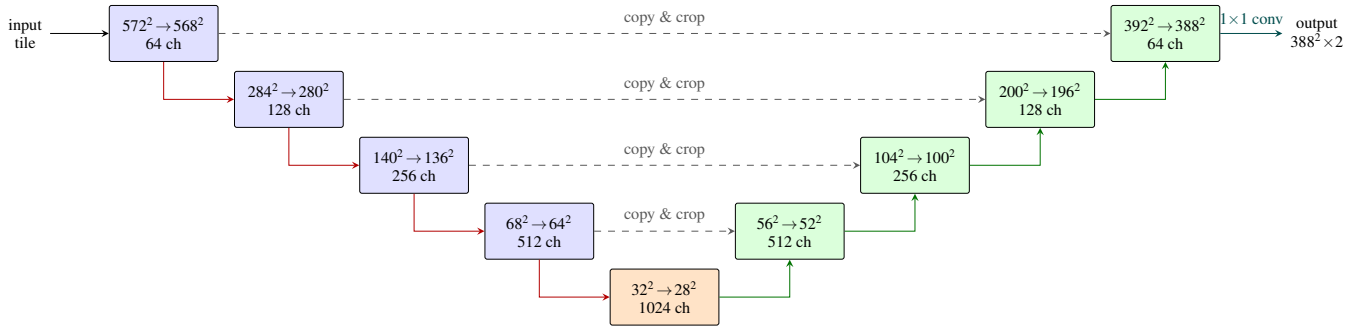


Figure 1. U-Net as implemented (size annotations are input^2 → after two valid convs^2 at each level). Red arrows: 2×2 max-pool; green arrows: 2×2 up-convolution; dashed: centre-cropped skip connection followed by concatenation. Each block is two 3×3 conv+ReLU layers.

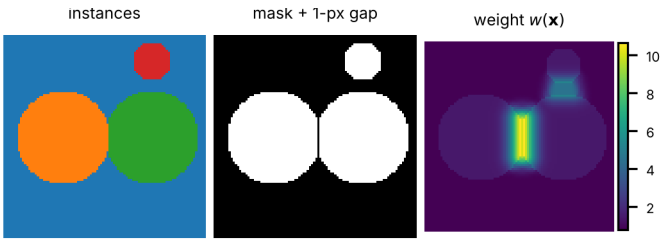


Figure 2. Output of my implementation on a synthetic example with three cells (two touching): instance labels, binary target with a 1-px separation border, and weight map $w(\mathbf{x})$ (Eq. 3). Weights reach about 10.7 in the gap and are ≈0.8 on distant background.

small, so the exponential term is close to w_0 and those few pixels dominate the loss; far from cells it vanishes. Touching cells are first separated by a one-pixel background border in the target mask, so the network can learn to leave a gap that later splits the objects (Fig. 2).

C. Initialisation and optimisation

In a deep network with several paths, poor initialisation can leave parts of the network inactive. For layers alternating convolution and ReLU, weights are drawn from $\mathcal{N}(0, 2/N)$, where N is the number of incoming connections of a neuron (e.g. $N = 3 \cdot 3 \cdot 64 = 576$) [5], which keeps feature-map variance near one. Since the unpadded output is smaller than the input, large tiles are preferred over large batches, so the batch is one image and a high momentum $\mu = 0.99$ is used:

$$v \leftarrow \mu v - \eta \nabla_\theta E, \qquad \theta \leftarrow \theta + v. \tag{4}$$

With $\mu = 0.99$ the update is effectively an average over roughly the last $1/(1-\mu) = 100$ samples.

D. Elastic deformation augmentation

Random elastic deformation is the paper's key tool for learning from few images, since deformation is the most common variation in tissue and can be simulated cheaply. A random displacement vector with components drawn from $\mathcal{N}(0, \sigma_d^2)$, $\sigma_d = 10$ px, is placed on each point of a coarse 3×3 grid; per-pixel displacements $\mathbf{D}(\mathbf{x})$ come from bicubic interpolation, and the image is resampled as $I'(\mathbf{x}) = I(\mathbf{x} + \mathbf{D}(\mathbf{x}))$. Dropout at the end of the contracting path acts as further implicit augmentation.

E. Overlap-tile inference

Large images are segmented tile by tile. Each 388×388 output block needs a 572×572 input window; missing context at the image border is extrapolated by mirroring. This avoids resolution being limited by GPU memory and gives seamless predictions.

F. Evaluation metric

The Cell Tracking Challenge SEG measure is the mean Jaccard index over ground-truth objects. For each ground-truth object g, let p^* be the predicted object that overlaps it most; then

$$J(g) = \begin{cases} \dfrac{|g \cap p^*|}{|g \cup p^*|} & \text{if } \dfrac{|g \cap p^*|}{|g|} > 0.5, \\ 0 & \text{otherwise,} \end{cases} \tag{5}$$

and SEG is the average of $J(g)$ over all ground-truth objects. I read the paper's "IOU" in Table 2 as this measure. As a second number I report plain foreground IoU.

III. Implementation Details

Framework. PyTorch, written from the paper; standard building blocks (`Conv2d`, `ConvTranspose2d`, SGD) are used [7]. No official or third-party U-Net repository was consulted. The paper's original implementation used Caffe [9].

Data. 2D gold-truth annotated frames of PhC-C2DH-U373 and DIC-C2DH-HeLa from the Cell Tracking Challenge [6]. The challenge's test labels are not public, so I train on the annotated frames of sequence 01 and test on those of sequence 02 (U373: 15 train / 19 test frames; HeLa: 9 / 9). Splitting by sequence avoids near-duplicate adjacent frames across the split. Images are rescaled to $[0, 1]$ using their 1st and 99th percentiles.

Training. Plain SGD, batch size 1, $\mu = 0.99$, learning rate $\eta = 10^{-3}$ reduced by 10× at 70% of training, gradient-norm clipping at 10, 6000 iterations, dropout 0.5, seed 0. The paper does not give a learning rate or schedule, so these are my choices and were not tuned. The loss is Eq. (2) averaged over pixels instead of summed. The up-convolution is a 2×2 stride-2 transposed convolution. Training uses fp16 autocast with a gradient scaler [8] for speed on a T4; the loss is computed in fp32.

Table 1. SEG ("IOU") scores. Paper values are from Table 2 of [1] on the hidden challenge test set; mine are on held-out annotated frames (see text).

Dataset	Ours SEG	Ours fg-IoU	Paper U-Net	Paper 2nd best
PhC-U373	0.772	0.736	0.9203	0.83
DIC-HeLa	0.164	0.826	0.7756	0.46

Sampling and augmentation. Each iteration draws a random 572×572 window from a mirror-padded training frame; the target is the central 388×388 region. Augmentation is elastic deformation (Sec. II-D, applied on the GPU to image, mask and weight map with the same field; nearest-neighbour for the mask), random flips, 90° rotations, and gray-value scaling and shift.

Inference. Overlap-tile prediction with mirror padding, averaged over 8 flip/rotation variants (test-time augmentation). The predicted foreground (probability > 0.5) is split into instances by connected components, and components smaller than 50 px are dropped. The reported model is the last iterate; test metrics were printed during training but were not used for model selection or early stopping.

Sanity checks. I verified on synthetic data that a 572^2 input gives a 388^2 output, that the model has 23 convolutional layers, that two touching discs receive a 1-px border and a weight of about 10 in the gap, and that the deformation keeps the mask binary. A practical note: the dataset TIFFs are LZW-compressed and need the `imagecodecs` package to be read.

IV. Results versus the Paper

Table 1 compares my numbers with Table 2 of the paper. *The comparison is indicative only*: the paper's scores come from the organisers' evaluation on a hidden test set after roughly ten hours of training on a Titan GPU, whereas mine come from held-out annotated frames of a different sequence, after 6000 iterations on a T4.

PhC-U373. The SEG score of 0.772 is 0.148 below the paper's U-Net and 0.058 below the paper's second-best entry (0.83). The result is therefore not a reproduction of the paper's number. Plausible contributors are the much shorter training, untuned hyperparameters, my instance-extraction step, and the different test set; I did not isolate their individual effects.

DIC-HeLa. Here the two metrics disagree sharply: foreground IoU is 0.826, but SEG is 0.164, far below even the paper's second-best entry (0.46). A high foreground IoU with a very low SEG means that foreground pixels are found correctly while individual cells are not. This is discussed in Sec. VI.

Reading the two metrics. SEG only iterates over ground-truth objects, so a spurious predicted object does not lower it, whereas foreground IoU does penalise false-positive area. The two numbers therefore need not be ordered, which is consistent with fg-IoU being slightly lower than SEG on PhC-U373. I did not check this explanation by counting false-positive objects.

Table 2. Ablation on PhC-U373 (single seed, 6000 iterations).

Variant	SEG	fg-IoU
Full (weight map + elastic)	0.772	0.736
No elastic deformation	0.764	0.732
No weight map	0.811	0.767

V. Ablation

To measure the contribution of the paper's two training ideas, I retrained on PhC-U373 with the same seed, schedule and split, removing elastic deformation or the weight map (the weight is then set to one everywhere; the 1-px separation border is kept).

Removing elastic deformation lowers SEG by 0.008, in the direction the paper argues but by a margin I cannot distinguish from run-to-run noise. Removing the weight map *raises* SEG by 0.039 and foreground IoU by 0.031. This is the opposite of what the paper's motivation predicts. With a single seed and 19 strongly correlated test frames from one sequence, I do not read it as evidence that the weight map is harmful. A hypothesis, which I did not test, is that PhC-U373 contains comparatively few touching cells, so the gap weighting offers little benefit there and may instead slow convergence in a 6000-iteration run. The weight map is designed for dense touching cells, as in DIC-HeLa, where I did not run the ablation.

VI. Failure Cases and What Did Not Work

HeLa instance collapse. The largest failure is DIC-HeLa SEG at 0.164 despite 0.826 foreground IoU. My leading hypothesis is that dense, touching HeLa cells are merged into one connected component by the post-processing: if the predicted 1-px gaps are missing or are bridged by the 0.5 threshold, several ground-truth cells map to one predicted object, and Eq. (5) gives zero for every ground-truth cell that the merged object does not cover by more than half. This is consistent with the symptom, but I have not verified it. Two checks would settle it: comparing the number of predicted components with the ground-truth cell count on test frames, and replacing connected components with a watershed on the probability map. If merging is the cause, the failure lies in my post-processing rather than in the network.

Gap not learned. Related to this, it is possible that the network does not predict the thin separating borders reliably after so few iterations. The weight map exists precisely to force this, and the paper trains far longer.

Shortfall on PhC-U373. Even on the easier dataset the model stays below the paper. I had no time or compute budget for tuning the learning rate, the schedule, the number of iterations or the threshold, so I cannot say how much of the gap is optimisation and how much is evaluation protocol.

Counter-intuitive ablation. The weight-map result (Sec. V) did not behave as expected, and my explanation is only a hypothesis.

Limits of the evidence. All results use a single seed, no confidence intervals, a small test set from a single sequence, and no HeLa ablations. Differences of a few hundredths in SEG should not be over-interpreted.

Minor. The first run failed because LZW-compressed TIFFs could not be decoded until `imagecodecs` was installed; this is a setup issue, not a modelling one.

VII. Conclusion

I re-implemented U-Net from the paper, including the unpadded architecture with cropped skip connections, the weight map, elastic deformation and overlap-tile inference, and ran a reduced-scale version of Table 2 on a free T4 GPU. The pipeline works end to end, but the numbers do not reach the paper's: 0.772 versus 0.920 SEG on PhC-U373, and a SEG of 0.164 on DIC-HeLa despite a foreground IoU of 0.826, which points to an instance-extraction problem that I have not yet confirmed. The ablation suggests a small benefit from elastic deformation and no benefit from the weight map on PhC-U373, but a single seed does not support firm conclusions. Natural next steps are to diagnose the HeLa merging, repeat runs over several seeds, train longer, and evaluate on the official test submission.

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241 (arXiv:1505.04597).

[2] D. C. Ciresan, L. M. Gambardella, A. Giusti, and J. Schmidhuber, "Deep neural networks segment neuronal membranes in electron microscopy images," in *Proc. NIPS*, 2012, pp. 2852–2860.

[3] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. CVPR*, 2015, pp. 3431–3440.

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. NIPS*, 2012, pp. 1106–1114.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. ICCV*, 2015, pp. 1026–1034.

[6] M. Maška *et al.*, "A benchmark for comparison of cell tracking algorithms," *Bioinformatics*, vol. 30, no. 11, pp. 1609–1617, 2014.

[7] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.

[8] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. ICLR*, 2018.

[9] Y. Jia *et al.*, "Caffe: Convolutional architecture for fast feature embedding," arXiv:1408.5093, 2014.